```
ACERTASTE 31 DE 36

ESTAS SON LAS QUE TIENES QUE EXPLICAR EN TU DOCUMENTO:

   · 7.0 / 0
      creiste que daba: 0
      en realidad da:   Infinity
      por que me equivoque: _____________________________

   · 0.0 / 0.0
      creiste que daba: 0
      en realidad da:   NaN
      por que me equivoque: _____________________________

   · (byte) 200
      creiste que daba: ...
      en realidad da:   -56
      por que me equivoque: _____________________________

   · 0.1 + 0.2
      creiste que daba: ...
      en realidad da:   0.30000000000000004
      por que me equivoque: _____________________________

   · Integer.MAX_VALUE + 1
      creiste que daba: ...
      en realidad da:   -2147483648
      por que me equivoque: _____________________________

################################################################################
```

7.0 / 0

Creí que daba: 0

En realidad da: Infinity

Por qué me equivoqué: pensé que marcaría error o daría cero. Pero en java al dividir un número decimal (double o float) entre cero, el sistema sigue el estándar de punto flotante en lugar de lanzar una excepción de enteros, arrojando el valor especial Infinity (infinito).

0.0 / 0.0

Creí que daba: 0

En realidad da: Infinity

Por qué me equivoqué: creí que daría cero. Pero pues se trata de una indeterminación matemática en aritmética por lo que Java devuelve NaN (Not a Number)

(byte) 200

Creí que daba: no supe

En realidad da: -56

Por qué me equivoqué:

Por qué me equivoqué: no tomé en cuenta el límite de almacenamiento. Un tipo byte en Java solo puede guardar valores desde -128 hasta 127. Al intentar meter el 200, ocurre un desbordamiento (overflow) que hace que la cuenta "dé la vuelta" y termine arrojando -56.

0.1 + 0.2

Creí que daba: ...

En realidad da: 0.30000000000000004

Por qué me equivoqué: cómo las computadoras almacenan los números decimales en formato binario, las fracciones no se representan de manera perfecta, lo que genera un pequeño error de precisión que resulta en 0.30000000000000004.

Integer.MAX_VALUE + 1

Creí que daba: ...

En realidad da: -2147483648

Por qué me equivoqué: no consideré el límite de los enteros de 32 bits. Cuando un número alcanza su valor máximo permitido (2147483647) y se le suma 1, ocurre un desbordamiento que hace que la numeración dé la vuelta y comience desde el extremo negativo, resultando en -2147483648.